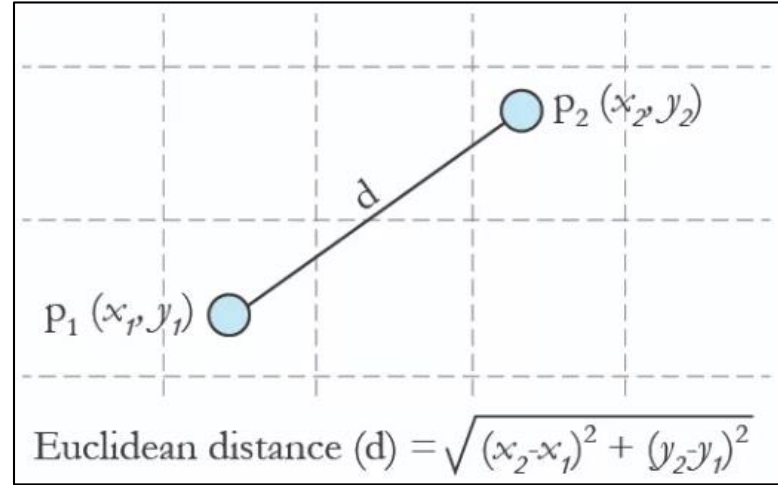






Xi'an Jiaotong-Liverpool University

西交利物浦大學

CPT111 Java Programming

Week 6 Exercise and Coursework-1

Introduction to Objects

Coding and Submission

- Coding in your NetBeans
 - Start with the skeleton code given in the course LMO
 - Add your own test cases
- Submitting into Learning Mall Quiz
 - Do not submit the whole class
 - Only submit the constructor or the method
 - read carefully the instructions
 - You can submit your own private helper method
 - but do not add another public methods

Exercise Week #6

- Complete the class Clock that represents time on a 24-hour clock, such as 00:00, 15:30, or 23:59
 - Time is measured in hours (00 – 23) and minutes (00 – 59)
 - Times are ordered from 00:00 (earliest) to 23:59 (latest)
 - A skeleton file Clock.java is given



Exercise #6.1 Clock Constructor 1

- Complete the first constructor of the class Clock
- It takes two arguments: h and m
 - and creates a new clock object whose initial time is h hours and m minutes
- Test cases:

```
Clock clock1 = new Clock(1, 0);
```

```
System.out.println(clock1);
```

→ 01:00

Exercise #6.2 Clock Constructor 2

- Complete the second constructor of the class Clock
- It takes one string argument: s
 - s is composed of two digits, followed by a colon, followed by two digits, so the format is HH:MM such as 02:30
 - it creates a new clock object whose initial time is HH hours and MM minutes

- Test cases:

```
Clock clock2 = new Clock("02:30");
```

```
System.out.println(clock2);
```

→ 02:30

Exercise #6.3 Clock toString

- Complete the method toString of the class Clock
- It returns a string representation of this clock, using the format HH:MM
 - that is, the format is the hours (2 digits), followed by a colon, followed by the minutes (2 digits), for example, 00:00 and 23:59
- Test cases:

```
Clock clock1 = new Clock(1, 0);
```

```
Clock clock2 = new Clock("02:30");
```

```
System.out.println(clock1);
```

→ 01:00

```
System.out.println(clock2);
```

→ 02:30

Exercise #6.4 Clock isEarlierThan

- Complete the method `isEarlierThan` of the class `Clock`
- It returns `true` if and only if the time on *this* clock (the current referenced object) is earlier than the time on *that* clock (in the argument)
- Test cases:

```
Clock clock1 = new Clock(1, 0);
```

```
Clock clock2 = new Clock("02:30");
```

```
System.out.println(clock1.isEarlierThan(clock2));    → true
```

Exercise #6.5 Clock tick

- Complete the method tick of the class Clock
- It adds 1 minute to the time on this clock
 - for example, one minute after 01:00 is 01:01; one minute after 23:59 is 00:00
- Test cases:

```
Clock clock1 = new Clock(1, 0);
```

```
System.out.println(clock1);
```

→ 01:00

```
clock1.tick();
```

```
System.out.println(clock1);
```

→ 01:01

Exercise #6.6 Clock tock

- Complete the method tock of the class Clock
- It adds delta minute(s) to the time on this clock, where delta is a positive integer
 - for example, 100 minutes after 02:30 is 04:10
- Note that must **not** use the method in CW1 #7.5 tick().
- Test cases:

```
Clock clock2 = new Clock("02:30");
```

```
System.out.println(clock2);
```

→ 02:30

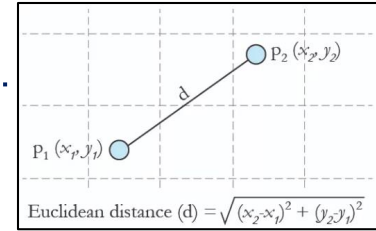
```
clock2.tock(100);
```

```
System.out.println(clock2);
```

→ 04:10

CW1 Week #6

- Complete the class EuclideanPoint that represents a point in the 2-dimensional space
 - It is represented by two double instance variables x and y
 - A skeleton file EuclideanPoint.java is given



CW1 #6.1 EuclideanPoint setCoordinates

- Complete the method setCoordinates of the class EuclideanPoint
- It sets both the x and y coordinates of the Euclidean point

- Test cases:

```
EuclideanPoint point2 = new EuclideanPoint();
```

```
point2.setCoordinates(7.0, 8.5);
```

```
System.out.println(point2.getX());           → 7.0
```

```
System.out.println(point2.getY());           → 8.5
```

CW1 #6.2 EuclideanPoint distanceTo

- Complete the method distanceTo of the class EuclideanPoint
- It returns the distance between this point and the other point in the parameter
- Test cases:

```
EuclideanPoint point1 = new EuclideanPoint(3.5, 5.2);
```

```
EuclideanPoint point2 = new EuclideanPoint();
```

```
point2.setCoordinates(7.0, 8.5);
```

```
double distance = point1.distanceTo(point2);
```

```
System.out.println(distance);           → 4.810405388322278
```

Thank you for your attention!

- This is the end of Week #6 Exercise and Coursework 1 Task Sheet